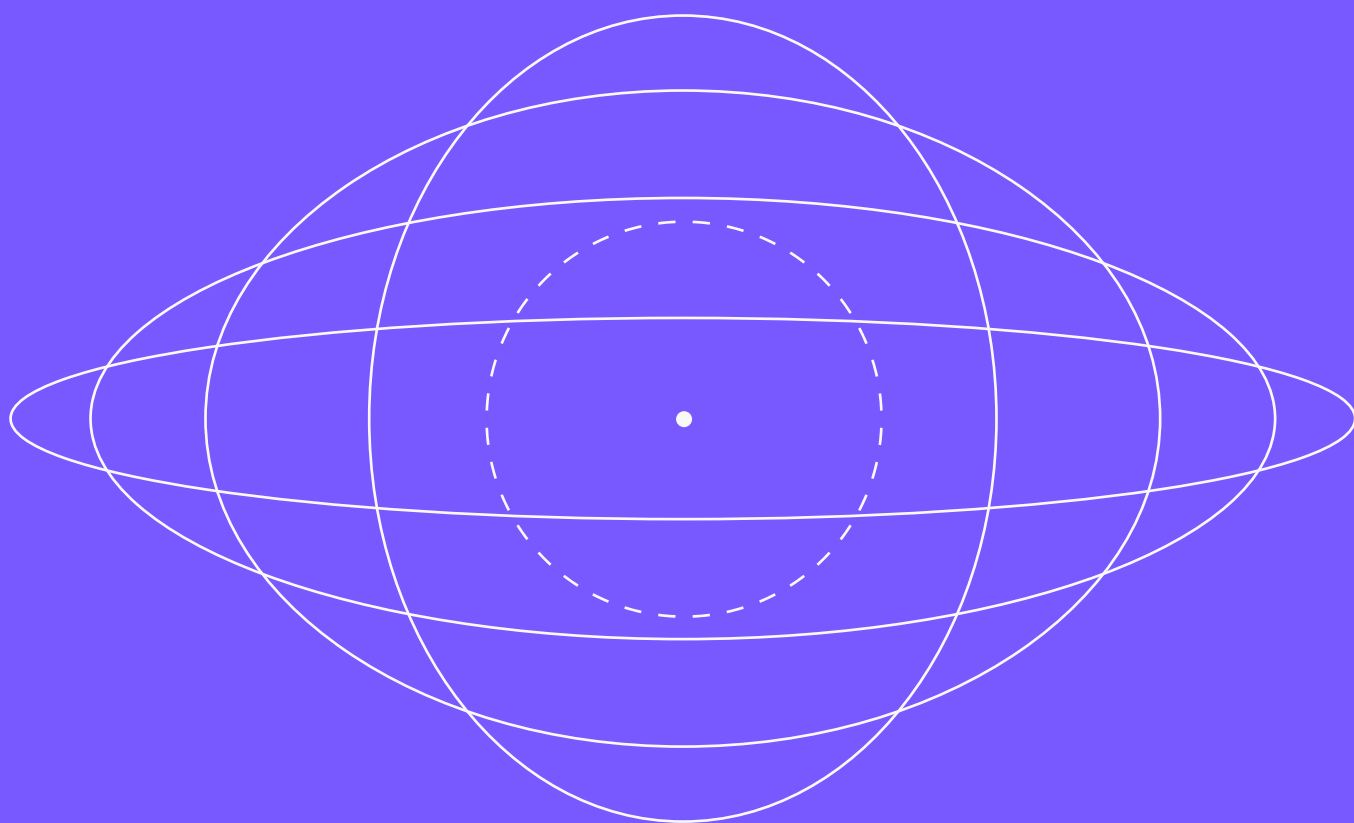


Автоматическое дифференцирование



МЕТОДЫ ВЫПУКЛОЙ ОПТИМИЗАЦИИ

СЕМИНАР

НЕДЕЛЯ 4

Нотация

$\mathbb{N}$	натуральные числа
$\mathbb{Z}$	целые числа
$\mathbb{R}$	вещественные числа
$\mathbb{R}_+$	неотрицательные вещественные числа
$\mathbb{R}_{++}$	положительные вещественные числа
$\overline{1, d}$	натуральные числа от 1 до d включительно
$:=$	по определению

КВАНТОРЫ И ЛОГИЧЕСКИЕ СИМВОЛЫ

$\forall$	квантор всеобщности
$\exists$	квантор существования
$\exists!$	квантор существования и единственности

ВЕКТОРНЫЕ ОБОЗНАЧЕНИЯ

$x \succ y$	покомпонентное сравнение векторов
$\langle x, y \rangle$	скалярное произведение в $\mathbb{R}^d$
$\ x\ _2$	евклидова норма
$\ x\ _p$	p -норма для $1 \leq p < \infty$
$\ x\ _\infty$	∞ -норма
$\nabla f(x)$	градиент функции f в точке x
$\nabla^2 f(x)$	гессиан функции f в точке x

$$x_i > y_i \text{ для всех } i$$

$$\langle x, y \rangle = \sum_{i=1}^d x_i y_i$$

$$\|x\|_2 = \sqrt{\langle x, x \rangle}$$

$$\|x\|_p = \left(\sum_{i=1}^d |x_i|^p \right)^{\frac{1}{p}}$$

$$\|x\|_\infty = \max_{i=1, d} |x_i|$$

$$(\nabla f(x))_i = \frac{\partial f}{\partial x_i}$$

$$(\nabla^2 f(x))_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

МАТРИЧНЫЕ ОБОЗНАЧЕНИЯ И НОРМЫ

$\langle X, Y \rangle$	скалярное произведение матриц	$\text{Tr}(X^T Y) = \sum_{i=1}^d \sum_{j=1}^n X_{ij} Y_{ij}$
$A \odot B$	поэлементное умножение матриц	$(A \odot B)_{ij} = A_{ij} \cdot B_{ij}$
$\ A\ $	индуцированная матричная норма	$\ A\ = \sup_{\ x\ =1} \ Ax\ $
$\ A\ _\infty$	бесконечная (строчная) норма	$\ A\ _\infty = \max_{i=1, \dots, d} \sum_{j=1}^n a_{ij} $
$\ A\ _1$	первая (столбцовая) норма	$\ A\ _1 = \max_{j=1, \dots, n} \sum_{i=1}^d a_{ij} $
$\ A\ _2$	вторая (евклидова, спектральная) норма	$\ A\ _2 = \sqrt{\lambda_{\max}(A^T A)}$
$\ A\ _F$	норма Фробениуса	$\ A\ _F = \sqrt{\sum_{i=1}^d \sum_{j=1}^n A_{ij}^2} = \sqrt{\text{Tr}(A^T A)}$

Симметричные матрицы и сравнения

$A \succ B$	$A - B$ положительно определена
$A \succeq B$	$A - B$ положительно полуопределена
$A \in \mathbb{S}^d$	$A = A^T$ (симметричная матрица)
$A \in \mathbb{S}_+^d$	A симметрична, и $A \succeq 0$
$A \in \mathbb{S}_{++}^d$	A симметрична, и $A \succ 0$

ПОЛЕЗНЫЕ НЕРАВЕНСТВА

Неравенство Йенсена:

$$f\left(\sum_{i=1}^n \alpha_i x_i\right) \leq \sum_{i=1}^n \alpha_i f(x_i), \quad (1)$$

где f — выпуклая, $\alpha_i \geq 0$, $\sum_{i=1}^n \alpha_i = 1$.

Неравенство Гёльдера:

$$|\langle x, y \rangle| \leq \|x\|_p \|y\|_q, \quad (2)$$

где $1 \leq p, q \leq \infty$, $\frac{1}{p} + \frac{1}{q} = 1$.

Неравенство Коши — Буняковского — Шварца:

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2. \quad (3)$$

Неравенства о средних:

$$\frac{n}{\sum_{i=1}^n \frac{1}{x_i}} \leq \left(\prod_{i=1}^n x_i\right)^{\frac{1}{n}} \leq \frac{1}{n} \sum_{i=1}^n x_i \leq \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}, \quad x_i > 0. \quad (4)$$

Неравенство между степенной функцией и экспонентой:

$$(1 - \alpha)^k \leq \exp(-\alpha k), \quad \alpha \in [0, 1]. \quad (5)$$

АСИМПТОТИЧЕСКИЕ ОБОЗНАЧЕНИЯ

$$\begin{aligned} f(n) \in \mathcal{O}(g(n)) & \quad \exists C > 0: f(n) \leq C \cdot g(n) \text{ для всех } n \in \mathbb{N} \\ f(n) \in \Omega(g(n)) & \quad \exists C > 0: f(n) \geq C \cdot g(n) \text{ для всех } n \in \mathbb{N} \\ f(n) \in \Theta(g(n)) & \quad \exists C_1, C_2 > 0: C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n) \text{ для всех } n \in \mathbb{N} \\ f(n) \in \tilde{\mathcal{O}}(g(n)) & \quad \exists \text{ полином } p: f(n) \in \mathcal{O}(g(n) \cdot p(\log n)) \text{ для всех } n \in \mathbb{N} \end{aligned}$$

ОБОЗНАЧЕНИЯ ДЛЯ РАСПРЕДЕЛЕНИЙ

$$U(\overline{1, n}) \quad \text{равномерное распределение на множестве } \overline{1, n}$$

Автоматическое дифференцирование

Контекст

Поговорим о том, как происходит подсчёт градиентов в реальной жизни. Чаще всего функции, с которыми приходится иметь дело на практике, представляют собой последовательность дифференцируемых параметрических преобразований. Её можно представить в виде **вычислительного графа**, где промежуточным вершинам соответствуют преобразования, входящим стрелкам — входные переменные, а выходным стрелкам — результат преобразования. **Автоматическое дифференцирование** — собирательное название способов вычисления производной сложной функции.

Задача лонгрида

Построить теорию автоматического дифференцирования и рассмотреть примеры.

Краткий план

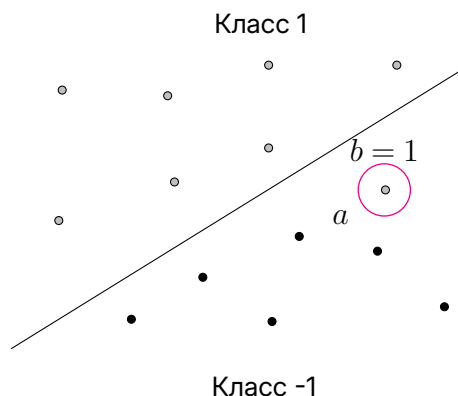
- [Обратное распространение ошибки](#)
- [Прямое распространение ошибки](#)
- [Умножение гессиана и якобиана на вектор](#)

Одна из классических задач машинного обучения — разделение выборки на два класса с помощью линейного классификатора с параметром x . Нужно найти такое значение коэффициентов прямой, чтобы как можно точнее разделить точки на классы согласно их меткам.

Предположим, что обучающий пример a имеет метку b из множества $\{-1, 1\}$.

В качестве функции потерь в таком случае традиционно выбирается логистическая функция потерь (log loss):

$$f(x) = \ln(1 + \exp(-b\langle x, a \rangle)).$$



ЗАМЕЧАНИЕ

Формула выше получена для одного обучающего примера. В реальных задачах приходится иметь дело с большими по размеру выборками и минимизировать усреднённую по всем объектам функцию потерь.

Большинство общепринятых методов оптимизации требует вычисления градиентов функции f по параметрам x . Именно здесь вычислительный граф становится удобным инструментом: он наглядно представляет все промежуточные преобразования. Распишем по шагам логистическую функцию потерь:

$$\begin{aligned} z(x) &= -b\langle x, a \rangle, \\ t(z) &= 1 + \exp(z), \\ r(t) &= \ln t. \end{aligned}$$

Итак, функция разложилась на последовательные параметрические преобразования:

$$f(x) = r(t(z(x))).$$

1. BACKWARD PROPAGATION

Начнём с дифференцирования цепочки последовательных параметрических преобразований — на её примере можно наглядно продемонстрировать принцип вычислений:

$$f(x) = u_n(u_{n-1}(\dots(u_1(x)))).$$

ЗАМЕЧАНИЕ

Для описания структуры графов будем использовать такие термины:

- **входы** — вершины, у которых есть только исходящие рёбра;
- **выходы** — вершины, у которых есть только входящие рёбра;
- **предки** вершины a — вершины, из которых есть путь в a ;
- **потомки** вершины a — вершины, в которые ведут пути из a .

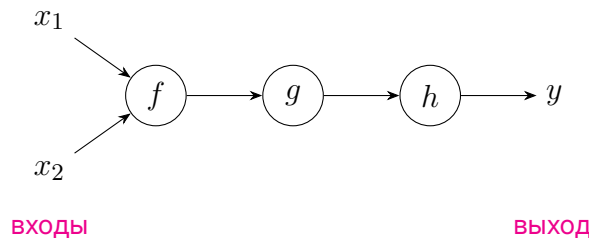


Рисунок 1. Пример линейного графа. Вершина g — потомок f и предок h

Функции такого вида имеют линейный вычислительный граф — в нём каждая вершина, кроме входов и выходов, имеет одного потомка и одного предка:

$$x \rightarrow u_1(x) \rightarrow u_2(u_1(x)) \rightarrow \dots \rightarrow u_n(u_{n-1}(\dots(u_1(x)))).$$

Вычисление значения функции по заданному входу называется **forward pass (прямым проходом)**. На этом этапе в вычислительном графе последовательно строятся промежуточные значения — результаты применения преобразований к предыдущим значениям слева направо. Но как же считать производные в таких графах? Ответ даёт правило вычисления дифференциала сложной функции. Рассмотрим одну конкретную вершину вида $u(x_1, \dots, x_d)$ и её дифференциал

$$du = J_u dx.$$

Каждый дифференциал dx_i можно расписать через дифференциал вершины-предка (если, конечно, x_i не соответствуют входам), двигаясь по рекурсии в графе от потомков к предкам. Получим итоговую формулу

$$\frac{\partial f}{\partial x}(x) = \frac{\partial u_1}{\partial x}(x) \cdot \frac{\partial u_2}{\partial u_1}(u_1) \cdot \dots \cdot \underbrace{\frac{\partial u_n}{\partial u_{n-1}}(u_{n-1})}_{\frac{\partial f}{\partial u_{n-1}}}. \quad (6)$$

Основная идея автоматического дифференцирования заключается в последовательном вычислении преобразований с накоплением значений вершин $u_1, \dots, u_n$ и последующем применении формулы (6) **справа налево**. Эта процедура называется **backward propagation (обратным распространением ошибки)**, в соответствии с тем, что градиент функции ошибок на выборке передаётся от вершин-потомков к предкам.

На рисунке 2 сплошными линиями приведён вычислительный граф для логистической функции потерь. Пунктирные линии обозначают обратное распространение ошибки. Первая вершина обратного прохода — это производная функции по самой себе — она всегда равна 1 и введена для удобства построения графа.

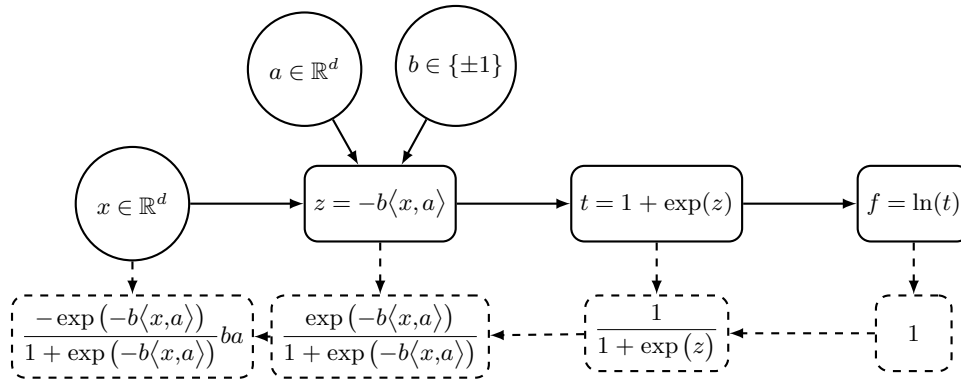


Рисунок 2. Граф вычислений для логистической функции потерь

В реальных задачах чаще приходится иметь дело с нелинейными вычислительными графами. Это означает, что формулу (6) нужно модифицировать в соответствии с правилами дифференцирования функций многих переменных. Пусть $u_1, \dots, u_n$ — вершины графа вычислений в топологическом порядке (то есть индексы предков не больше индексов потомков). Тогда общий алгоритм действий выглядит так:

1. Произвести forward pass и сохранить все значения u_i как функции от их вершин-предков.
2. Воспользоваться производной сложной функции



$$J_{f(u)} = J_f J_u$$

или, что то же самое, для всех $i = \overline{n-1, 1}$ посчитать

$$\frac{\partial f}{\partial u_i} = \sum_{u_j \in \text{потомки}(u_i)} \frac{\partial u_j}{\partial u_i} \frac{\partial f}{\partial u_j}.$$

ЗАДАЧА 1

Посчитай backward propagation для графа на рисунке 3, где $x \in \mathbb{R}^d$ — вход, $\omega \in \mathbb{R}^d$ и $b \in \mathbb{R}$ — обучаемые параметры, а $\lambda, t \in \mathbb{R}$ — фиксированные константы.

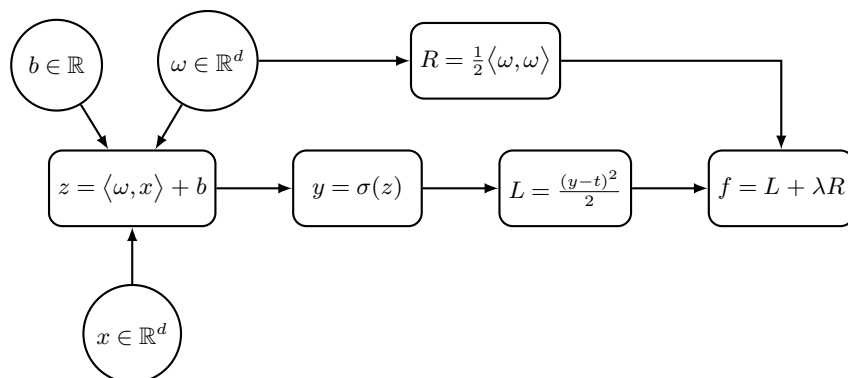


Рисунок 3. Граф вычислений логистической регрессии с функцией потерь MSE и L_2 -регуляризацией

РЕШЕНИЕ

$$\begin{aligned}
 \frac{\partial f}{\partial f} &= 1, \\
 \frac{\partial f}{\partial R} &= \frac{\partial f}{\partial R} \frac{\partial f}{\partial f} = \lambda, \\
 \frac{\partial f}{\partial L} &= \frac{\partial f}{\partial L} \frac{\partial f}{\partial f} = 1, \\
 \frac{\partial f}{\partial y} &= \frac{\partial L}{\partial y} \frac{\partial f}{\partial L} = (y - t) \frac{\partial f}{\partial L}, \\
 \frac{\partial f}{\partial z} &= \frac{\partial y}{\partial z} \frac{\partial f}{\partial y} = \sigma'(z) \frac{\partial f}{\partial y}, \\
 \frac{\partial f}{\partial \omega} &= \frac{\partial z}{\partial \omega} \frac{\partial f}{\partial z} + \frac{\partial R}{\partial \omega} \frac{\partial f}{\partial R} = x \frac{\partial f}{\partial z} + \omega \frac{\partial f}{\partial R}, \\
 \frac{\partial f}{\partial b} &= \frac{\partial z}{\partial b} \frac{\partial f}{\partial z} = \frac{\partial f}{\partial z}.
 \end{aligned}$$

Это достаточно простой пример, так как производные здесь считаются привычным способом. На практике в основном встречаются отображения, действующие в пространствах матриц и векторов. Чтобы лучше проиллюстрировать, как работает backward propagation в таком случае, вычислим его для полносвязной нейронной сети. Рассмотрим пример с поординатным применением нелинейной функции.

ЗАДАЧА 2

Найди градиент $\nabla f(x)$ и гессиан $\nabla^2 f(x)$ функции $f : \mathbb{R}^d \rightarrow \mathbb{R}$:

$$f(x) = h(g(x)),$$

где $g : \mathbb{R}^d \rightarrow \mathbb{R}^d : g(x) = \sin(x)$, $h : \mathbb{R}^d \rightarrow \mathbb{R} : h(u) = \sum_{i=1}^d u_i$.

РЕШЕНИЕ

Вспомним правило дифференцирования сложной функции:

$$J_f = J_h J_g \implies \nabla f = J_g^\top \nabla h.$$

Посчитаем матрицу Якоби функции вида $g(x) = \begin{pmatrix} g(x_1) \\ \vdots \\ g(x_d) \end{pmatrix}$. Получим

$$J_g = \text{diag}(g'(x_1), \dots, g'(x_d)) = \text{diag}(g'(x)) = J_g^\top.$$

При умножении J_g на вектор удобно пользоваться поэлементным умножением матриц (произведением Адамара):

$$(A \odot B)_{ij} = A_{ij} \cdot B_{ij}.$$

Результат умножения J_g на вектор y равен

$$J_g y = \begin{pmatrix} g'(x_1) \\ \vdots \\ g'(x_d) \end{pmatrix} \odot y = g'(x) \odot y.$$

Произведение Адамара вычисляется довольно быстро и легко поддаётся параллелизации. Теперь приступим непосредственно к примеру. Найдём J_g :

$$J_g = \text{diag}(\cos(x_1), \dots, \cos(x_d)) = \text{diag}(\cos(x)).$$

Теперь найдём $\nabla h(u)$:

$$\nabla h(u) = \begin{pmatrix} 1 \\ \vdots \\ 1 \end{pmatrix} = \mathbf{1}.$$

Тогда $\nabla f(x)$:

$$\nabla f(x) = J_g^\top \nabla h(u) = \cos(x) \odot \mathbf{1} = \cos(x).$$

Выпишем гессиан функции:

$$\nabla^2 f(x) = J_{\nabla f}^\top = \text{diag}(-\sin(x)).$$

ЗАДАЧА 3

Рассмотрим полносвязную нейронную сеть для задачи бинарной классификации с кросс-энтропийной функцией потерь:

$$\hat{y} = \sigma(\sigma(XW_1)W_2),$$

$$\mathcal{L}(y, \hat{y}) = - \sum_{i=1}^n [y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)],$$

где $X \in \mathbb{R}^{n \times d}$, $y \in \{0, 1\}^n$ — входы, $W_1 \in \mathbb{R}^{d \times k}$, $W_2 \in \mathbb{R}^k$ — обучаемые параметры.

Вычисли производные по параметрам W_1 , W_2 .

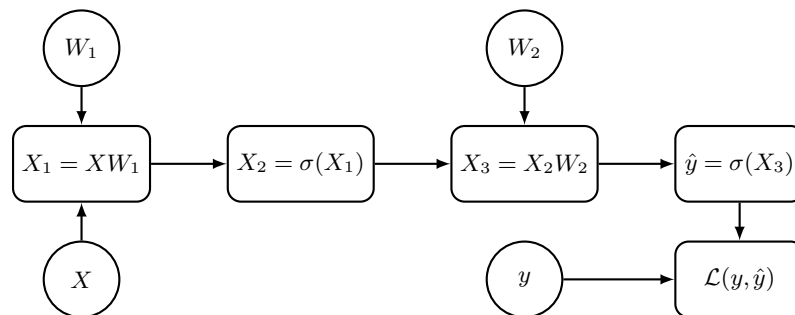


Рисунок 4. Прямой проход полносвязной нейросети с кросс-энтропийным лоссом

РЕШЕНИЕ

1. Найдём градиент функции потерь по $\hat{y}$ (все произведения покомпонентные):

$$\begin{aligned} d\mathcal{L}(\hat{y}) &= - \sum_{i=1}^n d(y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)) = \\ &= \sum_{i=1}^n \left(-\frac{y_i}{\hat{y}_i} d\hat{y}_i + \frac{1 - y_i}{1 - \hat{y}_i} d\hat{y}_i \right) = \sum_{i=1}^n \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)} d\hat{y}_i. \end{aligned}$$

Соответственно,

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i)}.$$

2. Вычислим производную по X_3 :

$$\frac{\partial \mathcal{L}}{\partial X_3} = \frac{\partial \hat{y}}{\partial X_3} \frac{\partial \mathcal{L}}{\partial \hat{y}} = \sigma'(X_3) \odot \frac{\partial \mathcal{L}}{\partial \hat{y}} = \sigma(X_3)(1 - \sigma(X_3)) \odot \frac{\partial \mathcal{L}}{\partial \hat{y}}.$$

3. Отсюда уже можно найти $\frac{\partial \mathcal{L}}{\partial W_2}$:

$$\frac{\partial \mathcal{L}}{\partial W_2} = \frac{\partial X_3}{\partial W_2} \frac{\partial \mathcal{L}}{\partial X_3} = X_2^\top \frac{\partial \mathcal{L}}{\partial X_3}.$$

4. Аналогично найдём $\frac{\partial \mathcal{L}}{\partial X_2}$:

$$\frac{\partial \mathcal{L}}{\partial X_2} = \frac{\partial X_3}{\partial X_2} \frac{\partial \mathcal{L}}{\partial X_3} = \frac{\partial \mathcal{L}}{\partial X_3} W_2^\top.$$

5. Снова ищем градиент для композиции с сигмоидой:

$$\frac{\partial \mathcal{L}}{\partial X_1} = \sigma'(X_1) \odot \frac{\partial \mathcal{L}}{\partial X_2} = \sigma(X_1)(1 - \sigma(X_1)) \odot \frac{\partial \mathcal{L}}{\partial X_2}.$$

6. Последний шаг проделываем по аналогии с 3-м:

$$\frac{\partial \mathcal{L}}{\partial W_1} = \frac{\partial X_1}{\partial W_1} \frac{\partial \mathcal{L}}{\partial X_1} = X^\top \frac{\partial \mathcal{L}}{\partial X_1}.$$

ЗАМЕЧАНИЕ

- Для подсчёта backward propagation необходимо хранить все промежуточные значения u_i на всех итерациях алгоритма. Это может быть существенным требованием к памяти, например, в больших нейронных сетях.
- Необязательно полностью считать производную $\frac{\partial u_j}{\partial u_i}$, важно уметь для каждого слоя быстро переходить от градиента по выходу к градиенту по входу. Об этом будет рассказано в секции 3.
- Отдельно стоит отметить нейронные сети, которые состоят из блоков. В каждом блоке backward propagation реализуется независимо от других блоков, после чего они механически, как кубики в конструкторе, собираются в большую сеть, которая работает как одно целое.
- Вычисление градиента можно представить через ещё один граф вычислений — то есть процедуру backward propagation можно запускать и по графу градиента. Так можно считать гессианы и производные высших порядков.

2. FORWARD PROPAGATION

Существует альтернативный подход, который может оказаться эффективнее backward propagation в ряде задач. Вместо применения формулы (6) справа налево можно вычислять производные по входу слоя и распространять производную к следующему слою. В финале такой процедуры в выходе графа накопится

значение производной. Формально, производная по параметрам x в таком случае считается следующим образом:

$$\frac{\partial u_i}{\partial x} = \sum_{u_j \in \text{родители}(u_i)} \frac{\partial u_j}{\partial x} \frac{\partial u_i}{\partial u_j}.$$

Эта процедура называется **forward propagation (прямое распространение ошибки)**. Поскольку производные при таком подходе вычисляются в процессе прямого прохода по вычислительному графу, эта процедура работает быстрее. Кроме того, она не требует хранить значения в узлах, достаточно лишь одного уровня графа.

ЗАДАЧА 4

Посчитай forward propagation для графа из задачи 1.

РЕШЕНИЕ

$$\begin{aligned} \frac{\partial R}{\partial \omega} &= \omega, & \frac{\partial z}{\partial b} &= 1, \\ \frac{\partial z}{\partial \omega} &= x, & \frac{\partial y}{\partial b} &= \frac{\partial z}{\partial b} \sigma'(z), \\ \frac{\partial y}{\partial \omega} &= \frac{\partial z}{\partial \omega} \sigma'(z), & \frac{\partial L}{\partial b} &= \frac{\partial y}{\partial \omega} \cdot (y - t), \\ \frac{\partial L}{\partial \omega} &= \frac{\partial y}{\partial \omega} \cdot (y - t), & \frac{\partial f}{\partial b} &= \frac{\partial L}{\partial b}. \\ \frac{\partial f}{\partial \omega} &= \frac{\partial L}{\partial \omega} + \lambda \frac{\partial R}{\partial \omega}. \end{aligned}$$

ЗАМЕЧАНИЕ

В forward propagation нужно хранить производные $\frac{\partial u_i}{\partial x}$, а в backward propagation — $\frac{\partial f}{\partial u_i}$. Если размерность входа x намного больше, чем размерность выхода $f(x)$, то на каждом шаге forward propagation нужно будет хранить и обсчитывать больше данных, чем в backward propagation. Если, наоборот, размерность выхода функции $f(x)$ намного больше, чем размерность входа x , то выгоднее использовать forward propagation.

3. УМНОЖЕНИЕ ГЕССИАНА НА ВЕКТОР И МАТРИЦЫ ЯКОБИ НА СТРОКУ

Пусть дана дважды непрерывно дифференцируемая функция $f: \mathbb{R}^d \rightarrow \mathbb{R}$. Покажем, как можно эффективно считать произведение её гессиана на произвольный вектор $u \in \mathbb{R}^d$:

$$\nabla^2 f(x) \cdot u.$$

Считать полный гессиан и умножать его на вектор неэффективно с алгоритмической точки зрения, особенно при большой размерности d . Но заметим, что

$$d\langle \nabla f(x), u \rangle = \langle J_{\nabla f} dx, u \rangle = \langle \nabla^2 f(x) \cdot u, dx \rangle = \langle \nabla g(x), dx \rangle,$$

где $g(x) = \langle \nabla f(x), u \rangle$. Таким образом, вместо полного гессиана можно найти градиент функции вида $g(x)$, с которым могут справиться библиотеки автоматического дифференцирования `jax/autograd/pytorch/tensorflow` без необходимости хранения огромной матрицы.

4. ЕСЛИ КРАТКО

- Вычислительный граф — представление функции $f(x)$ в виде композиции простых преобразований с промежуточными вершинами u_i .
- Вычисление значения функции по заданному входу называется forward pass.
- Вычисление градиента сложной функции с помощью [backward propagation](#):

$$\frac{\partial f}{\partial u_i} = \sum_{u_j \in \text{потомки}(u_i)} \frac{\partial u_j}{\partial u_i} \frac{\partial f}{\partial u_j}.$$

- Вычисление градиента сложной функции с помощью [forward propagation](#):

$$\frac{\partial u_i}{\partial x} = \sum_{u_j \in \text{родители}(u_i)} \frac{\partial u_j}{\partial x} \frac{\partial u_i}{\partial u_j}.$$